ĐỀ XUẤT PHƯƠNG ÁN KIẾN TRÚC & CẢI TIẾN HỆ THỐNG

IPGS Remote Control (CCU ↔ ZCU) — Version 1.2+

Mã tài liệu: `PROPOSAL-remote-control-improvements-2026-10-31`

Ngày lập: 2026-10-31

Tác giả: Tech Lead & Senior Developer

Căn cứ: Báo cáo Rà soát kỹ thuật `REVIEW-remote-control-full-audit-2026-10-31` và Nghiên cứu đối chuẩn `RESEARCH-rustdesk-2026-09-29`.

Phê duyệt đề xuất: Engineering Manager & CTO

---

1. MỤC TIÊU CHIẾN LƯỢC & NGUYÊN TẮC KIẾN TRÚC

Hệ thống Remote Control phục vụ các môi trường vận hành công nghiệp khắc nghiệt:

- Thiết bị đầu cuối ZCU đa dạng về phần cứng: từ các máy tính công nghiệp Intel Celeron J3455/J6412 đến các bo mạch ARM64 nhúng Rockchip RK3568/RV1126 có tài nguyên RAM (2GB) và CPU hạn chế.
- Kênh truyền mạng nội bộ (LAN): có thể biến thiên độ trễ hoặc bị nghẽn cục bộ.
- Yêu cầu vận hành 24/7: tuyệt đối không được rò rỉ RAM (Memory Leak), không treo tiến trình và bảo vệ tuyệt đối an toàn truy cập (Zero-Trust/Encryption).

4 Nguyên tắc kiến trúc cốt lõi:

1. **Zero-Copy & Buffer Reuse:** Loại bỏ hoàn toàn việc cấp phát bộ nhớ động trong vòng lặp streaming (15–30 fps). Tái sử dụng vùng đệm cố định.
2. **Deterministic Threading:** Đảm bảo luồng nhận gói tin mạng (TCP Receiver), luồng inject phần cứng (X11) và luồng hiển thị giao diện (Avalonia UI Thread) hoàn toàn độc lập, không gây nghẽn chéo và không cấp phát task thừa trên ThreadPool.
3. **Hardened Security:** Mã hóa kênh truyền mặc định, bảo vệ thông tin xác thực (không lộ secret trong process table hay network sniff).
4. **Resilient Testing:** Mọi module giao thức cốt lõi phải có Unit Test bao phủ 100% các nhánh biên.

---

2. DANH MỤC ĐỀ XUẤT CẢI TIẾN PHÂN TẦNG

GÓI P1: CHỐT CHẶN CHẤT LƯỢNG RELEASE (BẮT BUỘC TRƯỚC KHI PHÁT HÀNH)

Đề xuất 1.1: Triệt tiêu phình LOH 124 MB/s bằng Kỹ thuật Ping-Pong Double-Buffering

- **Vấn đề giải quyết:** Mã lỗi **M1** (124 MB/s rác LOH trên `RemoteScreenViewModel`).
- **Giải pháp kiến trúc:**
  1. Duy trì 2 đối tượng `WriteableBitmap` (`_backBuffer` và `_frontBuffer`) được khởi tạo một lần duy nhất theo độ phân giải hiện tại.

2. Khi nhận frame JPEG mới, giải mã trực tiếp vào `_backBuffer.Lock().Address` mà không qua mảng trung gian `byte[] pixData`.
3. Hoán đổi con trỏ (Swap Buffers) và gửi `_frontBuffer` lên UI hiển thị thông qua `Dispatcher.UIThread.Post`.
4. Chỉ cấp phát lại cặp buffer khi kích thước màn hình ZCU thực sự thay đổi (RandR resize/rotate).

- **Hiệu quả đạt được:**
  - Cắt giảm **100% việc cấp phát bộ nhớ trên Large Object Heap (LOH)** trong quá trình xem remote.
  - RAM của `CcuUI` duy trì phẳng ổn định ở mức ~110–130 MB, triệt tiêu hoàn toàn hiện tượng khựng hình do Gen 2 GC.

Đề xuất 1.2: Tái sử dụng SKBitmap & Kiểm soát Unmanaged Heap trên ZcuAgent

- **Vấn đề giải quyết:** Mã lỗi **M2** (8.3 MB malloc/free unmanaged mỗi frame gây phân mảnh RAM Linux).
- **Giải pháp kiến trúc:**
  1. Lưu trữ một instance `SKBitmap` tái sử dụng trong `JpegEncoder`. Cập nhật con trỏ pixel qua `SKBitmap.InstallPixels` trỏ trực tiếp vào `_pixelBuffer` của `X11ScreenCapturer` mà không cấp phát unmanaged buffer mới.
  2. Kích hoạt lệnh P/Invoke `malloc_trim(0)` định kỳ (mỗi 60 giây hoặc khi client ngắt kết nối) theo đúng bài học kinh nghiệm tại `GOTCHAS.md` (G002) để ép kernel Linux thu hồi unmanaged arena.
- **Hiệu quả đạt được:**
  - VmRSS của `ZcuAgent` ổn định tuyệt đối ở mức **< 65 MB RAM** trên môi trường Linux Ubuntu.

Đề xuất 1.3: Cập nhật bản vá lỗ hổng bảo mật SSH.NET

- **Vấn đề giải quyết:** Mã lỗi **S1** (Cảnh báo compiler NU1903 High Severity Vulnerability: GHSA-mggc-4xg6-vcxf, GHSA-q939-rpr3-3284).
- **Giải pháp:**
  - Nâng cấp package `SSH.NET` trong `IPGS.RemoteControl.CcuClient.csproj` lên phiên bản ổn định mới nhất đã khắc phục triệt để lỗ hổng xác thực và DoS.

Đề xuất 1.4: Tối ưu Input Queue & Cơ chế Coalescing chuột trên Agent

- **Vấn đề giải quyết:** Mã lỗi **C1** & **C3** (Quá tải ThreadPool và trôi chuột khi X11 lag).
- **Giải pháp kiến trúc:**
  1. Thay thế `Channel<Action>` bằng một cấu trúc dữ liệu nhẹ `struct InputCommand` (chứa opcode, x, y, button, keysym) không cấp phát heap closure.
  2. Thực thi tuần tự trực tiếp trên vòng lặp consumer mà không bắn thêm `Task.Run` trung gian cho từng coordinate.
  3. Áp dụng cơ chế **Coalescing**: nếu trong hàng đợi đang có sự kiện di chuột `MouseMove` chưa kịp xử lý và nhận thêm sự kiện di chuột mới, agent tự động cập nhật tọa độ cuối cùng và bỏ qua các điểm trung gian.
- **Hiệu quả đạt được:**
  - Giảm 95% số lượng Task trên ThreadPool; con trỏ chuột bám dính thời gian thực, không bị trôi hay trễ.

Đề xuất 1.5: Sửa vi phạm luồng UI Thread trong RemoteScreenControl

- **Vấn đề giải quyết:** Mã lỗi **C2** (Gọi `PasteStatus?.Invoke(null)` từ ThreadPool).
- **Giải pháp:**

- Bọc lời gọi callback qua `Dispatcher.UIThread.Post(() => PasteStatus?.Invoke(null))` đảm bảo an toàn tuyệt đối 100% cho UI Thread Avalonia.

Đề xuất 1.6: Xây dựng Project Unit Test Suite (IPGS.RemoteControl.Tests)

- **Vấn đề giải quyết:** Mã lỗi **T1** (Thiếu kiểm thử tự động).
- **Giải pháp:**
  - Tạo project `IPGS.RemoteControl.Tests` (xUnit + FluentAssertions).
  - Viết test suite bao phủ toàn bộ:
    - `MessageCodecTests`: Kiểm thử serialize/deserialize tất cả các MessageType (kể cả boundary conditions, corrupt payload, BigEndian conversion).
    - `ShellQuoteTests`: Kiểm thử các vector tấn công shell injection (`$()`, `;`, `&&`, backticks, quotes).
    - `PhysicalKeyMapperTests`: Kiểm thử ánh xạ phím Windows VirtualKey ↔ X11 keycode.
    - `ScreenCoordMapperTests`: Kiểm thử thuật toán scale và letterbox uniform.

GÓI P2: NÂNG CAO ĐỘ TIN CẬY & BẢO MẬT (SPRINT KẾ TIẾP)

Đề xuất 2.1: Chuẩn hóa bảo mật Sudo Password qua STDIN Stream

- **Vấn đề giải quyết:** Mã lỗi **S2** (Lộ mật khẩu sudo trong biến môi trường của `RemoteAppInstallService`).
- **Giải pháp:**
  - Xóa bỏ hoàn toàn biến `KIOSK_SUDO_PASS` khỏi command line.
  - Sử dụng chung mô hình kênh STDIN (`SshClient.CreateShellStream` hoặc `sudo -S -p ''`) như đã triển khai thành công tại `RemoteCommandWindow`. Mật khẩu được ghi trực tiếp vào stdin stream và xóa khỏi bộ nhớ ngay sau đó.

Đề xuất 2.2: Tối ưu Zero-Copy Upload File Chunking

- **Vấn đề giải quyết:** Mã lỗi **M3** (`.ToArray()` sinh rác khi chia chunk upload 64 KB).
- **Giải pháp:**
  - Cập nhật hàm `MessageCodec.WriteMessageAsync` hỗ trợ nhận `ReadOnlyMemory<byte>` thay vì chỉ nhận `byte[]`.
  - Đọc file trực tiếp vào buffer nội bộ và gửi qua socket, loại bỏ hoàn toàn việc cấp phát mảng 64 KB mới cho mỗi chunk.

Đề xuất 2.3: Bổ sung Certificate Fingerprint Pinning cho TLS

- **Vấn đề giải quyết:** Mã lỗi **S3** (Nguy cơ MitM mạng LAN).
- **Giải pháp:**
  - Khi CCU kết nối thành công lần đầu tới ZCU, lưu trữ mã băm SHA-256 của chứng chỉ SSL vào `ComputerProfile`.
  - Trong các lần kết nối tiếp theo, `SslStream.RemoteCertificateValidationCallback` sẽ so khớp vân tay chứng chỉ với mã băm đã lưu. Cảnh báo ngay cho quản trị viên nếu phát hiện chứng chỉ bị thay đổi bất thường.

Đề xuất 2.4: Khắc phục công thức giám sát CPU trên HealthMonitorWindow

- **Vấn đề giải quyết:** Mã lỗi **U1** (Báo cáo sai lệch hiệu năng hệ thống).
- **Giải pháp:**

- Thay đổi biểu thức chính quy (Regex) để trích xuất chỉ số `id` (Idle CPU) từ lệnh `top -b -n 1`:

`Total CPU Utilization % = 100.0 - Idle%`

- Đảm bảo phản ánh chính xác cả tải người dùng (`us`), tải nhân (`sy`) và nghẽn ổ đĩa/I-O (`wa`).

GÓI P3: TIẾN HÓA CÔNG NGHỆ DÀI HẠN (CHUẨN HÓA THEO RUSTDESK)

Đề xuất 3.1: Hỗ trợ Wayland Native Capture qua PipeWire Portal

- **Mục tiêu:** Cho phép ZcuAgent chạy mượt mà trên Ubuntu 22.04 / 24.04 sử dụng Wayland (GNOME Display Manager) mà không cần cấu hình ép về X11.
- **Giải pháp:**
  - Học hỏi từ `src/server/wayland.rs` của RustDesk: sử dụng giao tiếp D-Bus tới `org.freedesktop.portal.ScreenCast` để lấy PipeWire stream, và `org.freedesktop.portal.RemoteDesktop` để inject phím chuột mà không đòi hỏi quyền root.

Đề xuất 3.2: Đồng bộ hóa hình dạng con trỏ chuột (Cursor Shape Sync)

- **Mục tiêu:** Client hiển thị chính xác trạng thái con trỏ chuột từ xa (con trỏ chỉ tay, con trỏ gõ chữ, con trỏ quay chờ tải, con trỏ kéo giãn cửa sổ).
- **Giải pháp:**
  - Bổ sung `MessageType.CursorShape (0x44)`. ZcuAgent lắng nghe sự kiện `XFixesSelectCursorInput` và gửi ảnh bitmap con trỏ kèm hotspot về CCU để render.

Đề xuất 3.3: Dọn dẹp nợ kỹ thuật XAML Obsolete

- **Giải pháp:** Chuyển đổi toàn bộ 32 thuộc tính `TextBox.Watermark` sang `PlaceholderText` theo đúng chuẩn Avalonia 12.

3. MA TRẬN ĐÁNH GIÁ & KẾ HOẠCH TRIỂN KHAI

Gói	Hạng mục đề xuất	Độ ưu tiên	Effort	Rủi ro kỹ thuật	Lợi ích mang lại
P1	1.1 Ping-Pong Double Buffering	P1	1.5 ngày	Thấp (đã có WriteableBitmap Lock)	Triệt tiêu 124 MB/s LOH rác, app mượt mà 100%
P1	1.2 Tái sử dụng SKBitmap & malloc_trim	P1	0.5 ngày	Thấp	Khống chế RAM Agent < 65MB, không phân mảnh
P1	1.3 Nâng cấp bản vá SSH.NET	P1	0.5 ngày	Rất thấp	Triệt tiêu lỗ hổng bảo mật mức cao
P1	1.4 Input Queue struct & Coalescing	P1	1.0 ngày	Trung bình (cần test kỹ chuột)	Giảm tải ThreadPool, chuột không trôi
P1	1.5 Fix luồng UI Thread PasteStatus	P1	0.2 ngày	Rất thấp	Ngăn chặn nguy cơ crash ngẫu nhiên
P1	1.6 Xây dựng bộ Unit Test Suite	P1	1.5 ngày	Thấp	Bảo vệ chất lượng chống hồi quy
P2	2.1 Sudo password qua STDIN	P2	0.5 ngày	Thấp	Bảo mật tuyệt đối tài khoản root
P2	2.2 Upload Chunk Zero-Copy	P2	0.5 ngày	Thấp	Tối ưu hóa truyền tệp tin lớn
P2	2.3 Certificate Fingerprint Pinning	P2	0.5 ngày	Thấp	Ngăn chặn tấn công MitM
P2	2.4 Sửa công thức CPU Health Monitor	P2	0.2 ngày	Rất thấp	Giám sát sức khỏe chính xác 100%
P3	3.1 Wayland PipeWire Capture	P3	5.0 ngày	Cao (D-Bus & PipeWire C#	Tương thích toàn diện Linux hiện đại

Gói	Hạng mục đề xuất	Độ ưu tiên	Effort	Rủi ro kỹ thuật	Lợi ích mang lại
				binding)	
P3	3.2 Cursor Shape Synchronization	P3	2.0 ngày	Trung bình	Trải nghiệm người dùng chuyên nghiệp
P3	3.3 Chuẩn hóa XAML Avalonia 12	P3	0.5 ngày	Rất thấp	Mã nguồn sạch sẽ, không cảnh báo

Đề xuất được đệ trình lên Engineering Manager và CTO để phê duyệt lộ trình thực thi.